BIHE Website + Admin Panel Integration Guide

Project: Bapuji Institute of Hi-Tech Education (BIHE) Website
Stack: Next.js 15, React 19, TypeScript
Purpose: Step-by-step guide for integrating the public website with a custom admin panel
Audience: Website team + Admin panel team
Version: 1.0 | June 2026

Table of Contents

---

1. Overview

Current State

The BIHE website is a **static Next.js marketing site**. All content is hardcoded in TypeScript files:

Content Type	Current Location
Page copy & tables	`src/lib/*-content.ts`
Navigation	`src/lib/*-submenu.ts`
Images	`public/images/`
PDFs	`public/documents/`

There is **no database, API, auth, or CMS** today.

Target Architecture

```
Admin Panel (/admin)  →  API (/api/*)  →  Database (PostgreSQL)  →
Public Website (/)
```

Integration Goal

- Admin team **writes** content (faculty, news, announcements, PDFs)
- Website team **reads** content via API
- Both work in **one Git repo** and **one Cursor workspace**
- Changes go live without full redeploy (ISR + revalidation)

One-Line Summary

One Git repo → both open in Cursor → admin owns `/admin` + `/api` + DB → website fetches from API → merge via branches/PRs → deploy one Vercel project.

---

2. Phase 1 — One Place for Everyone (Day 1)

Step 1: Create One Git Repository

```
cd "BIHE Website"
git init
git add .
git commit -m "Initial BIHE public website"
```

Create a repository on GitHub or GitLab, then:

```
git remote add origin https://github.com/YOUR-ORG/bihe-website.git
git branch -M main
git push -u origin main
```

Both teammates: clone the same repository.

Step 2: Share the Cursor Workspace

Person	Action
Everyone	Cursor → File → Open Folder → select cloned `bihe-website`
Everyone	Run `npm install`
Everyone	Run `npm run dev` → open http://127.0.0.1:3000

Important: Cursor does not sync code between machines. **Git is the sync tool.**

Step 3: Agree on Folder Ownership

```
BIHE Website/
├── src/
│   ├── app/
│   │   ├── (public pages)        ← Website dev
│   │   ├── admin/                ← Admin dev ONLY
│   │   └── api/                  ← Admin dev ONLY
│   ├── components/
│   │   ├── landing/              ← Website dev
│   │   ├── administration/       ← Website dev
│   │   └── admin/                ← Admin dev ONLY
│   └── lib/
│       ├── types/                ← BOTH (shared contracts)
│       ├── content/              ← Website dev (fetch helpers)
│       └── db/                   ← Admin dev
```

Developer	Works In
Website person	`src/app/**/page.tsx`, `src/components/landing/`, `src/components/administration/`, `src/lib/*-content.ts`
Admin person	`src/app/admin/`, `src/app/api/`, `src/components/admin/`, `src/lib/db/`
Both	`src/lib/types/`

---

3. Phase 2 — Shared Contract (Day 1–2)

Step 4: Create Shared Types

Create file: `src/lib/types/content.ts`

```ts
export type FacultyMember = {
  id: string;
  name: string;
  designation: string;
  qualification: string;
  experience: string;
  department: "b-com" | "bca";
  sortOrder: number;
};

export type ProgramRow = {
  id: string;
  level: string;
  programName: string;
  duration: string;
  intake: string;
  department: "b-com" | "bca";
};

export type NewsItem = {
  id: string;
  title: string;
  date: string;
  excerpt: string;
  image?: string;
  published: boolean;
};

export type Announcement = {
  id: string;
  message: string;
  link?: string;
  active: boolean;
};
```

Rule: Admin saves this shape. Website reads this shape. No custom formats per developer.

Step 5: Git Branch Workflow

```
# Website developer
git checkout -b feature/website-b-com
git pull origin main

# Admin developer
git checkout -b feature/admin-panel
git pull origin main
```

Daily routine for both:

```
git pull origin main
# ... do your work ...
git add .
git commit -m "Describe your change"
git push origin your-branch
# Open Pull Request → merge to main
```

---

4. Phase 3 — Backend Layer (Day 2–4)

Step 6: Add Database (PostgreSQL)

Use **Neon** (neon.tech) or **Supabase** (supabase.com).

1. Create a new project
2. Copy the connection string
3. Create `.env.local` (never commit this file):

```
DATABASE_URL="postgresql://..."
ADMIN_SECRET="long-random-secret"
NEXTAUTH_SECRET="another-long-secret"
NEXTAUTH_URL="http://127.0.0.1:3000"
```

Add to `.gitignore`:

```
.env.local
.env*.local
```

Step 7: Install Backend Packages

```
npm install @neondatabase/serverless drizzle-orm
npm install -D drizzle-kit
npm install next-auth@beta bcryptjs
npm install -D @types/bcryptjs
```

Step 8: Create Database Schema

Create file: `src/lib/db/schema.ts`

```ts
import { pgTable, text, integer, boolean, timestamp } from "drizzle-orm/pg-core";

export const faculty = pgTable("faculty", {
  id: text("id").primaryKey(),
  name: text("name").notNull(),
  designation: text("designation").notNull(),
  qualification: text("qualification").notNull(),
  experience: text("experience").notNull(),
  department: text("department").notNull(),
  sortOrder: integer("sort_order").default(0),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const programs = pgTable("programs", {
  id: text("id").primaryKey(),
  level: text("level").notNull(),
  programName: text("program_name").notNull(),
  duration: text("duration").notNull(),
  intake: text("intake").notNull(),
  department: text("department").notNull(),
});

export const announcements = pgTable("announcements", {
  id: text("id").primaryKey(),
  message: text("message").notNull(),
  link: text("link"),
  active: boolean("active").default(true),
});
```

Run database migrations once (admin developer).

Step 9: Create API Routes

```
src/app/api/
├── faculty/
│   ├── route.ts          GET (public) + POST (admin)
│   └── [id]/route.ts     PUT + DELETE (admin)
├── programs/
│   └── route.ts
├── announcements/
│   └── route.ts
└── auth/
    └── [...nextauth]/route.ts
```

Example public read endpoint — `src/app/api/faculty/route.ts` :

```ts
import { NextResponse } from "next/server";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const department = searchParams.get("department") ?? "b-com";
  // const rows = await db.select().from(faculty).where(...)
  return NextResponse.json({ data: [] });
}
```

Protect all write routes (POST, PUT, DELETE) with admin session check.

---

5. Phase 4 — Admin Panel (Day 3–7)

Step 10: Create Admin Routes

```
src/app/admin/
├── layout.tsx              ← sidebar + auth guard
├── page.tsx                ← dashboard
├── login/page.tsx
├── faculty/
│   ├── page.tsx            ← list
│   ├── new/page.tsx
│   └── [id]/edit/page.tsx
├── programs/page.tsx
└── announcements/page.tsx
```

URLs:

URL	Purpose
http://127.0.0.1:3000/	Public website
http://127.0.0.1:3000/admin	Admin panel
http://127.0.0.1:3000/api/faculty	API

Step 11: Add Authentication

1. Create admin users table (email + hashed password)
2. Set up NextAuth at `src/app/api/auth/[...nextauth]/route.ts`
3. Protect `/admin/*` in `src/app/admin/layout.tsx`
4. Redirect unauthenticated users to `/admin/login`
5. **Never** add admin links to the public `Header.tsx`

Step 12: Build Admin CRUD (Priority Order)

Order	Module	Admin Page	API
1	Announcements	`/admin/announcements`	`/api/announcements`
2	B.Com programme table	`/admin/programs`	`/api/programs`
3	Faculty (B.Com, BCA)	`/admin/faculty`	`/api/faculty`
4	News & events	`/admin/news`	`/api/news`
5	PDF uploads	`/admin/documents`	`/api/documents`

Start with announcements and programme table to prove the full loop works.

---

6. Phase 5 — Connect Website to API (Day 5–10)

Step 13: Create Content Fetch Layer

Create file: `src/lib/content/faculty.ts`

```
import type { FacultyMember } from "@/lib/types/content";

export async function getFacultyByDepartment(
  department: "b-com" | "bca"
): Promise<FacultyMember[]> {
  const base = process.env.NEXT_PUBLIC_SITE_URL ??
"http://127.0.0.1:3000";
  const res = await fetch(`${base}/api/faculty?
department=${department}`, {
    next: { revalidate: 300 },
  });
  if (!res.ok) return [];
  const json = await res.json();
  return json.data ?? [];
}
```

Create similar files for programs, announcements, and news.

Step 14: Replace Hardcoded Content (Page by Page)

Before (static):

```
import { B_COM_PROGRAM_TABLE } from "@/lib/b-com-admin-content";
```

After (dynamic with fallback):

```
import { getProgramTable } from "@/lib/content/programs";
import { B_COM_PROGRAM_TABLE } from "@/lib/b-com-admin-content";

const table = await getProgramTable("b-com");
const data = table.length ? table : B_COM_PROGRAM_TABLE.rows;
```

Migration order:

1. `AnnouncementBar.tsx`
2. `BComFacultySection.tsx` / programme table
3. `NewsEventsSection.tsx`
4. `courses-content.ts`
5. Other `*-content.ts` files (last)

Keep static files as fallback until admin data is verified.

Step 15: Live Updates After Admin Saves

In admin API, after successful save:

```
import { revalidatePath } from "next/cache";

revalidatePath("/b-com");
revalidatePath("/");
```

Result: admin saves → public page updates within seconds (no full redeploy).

---

7. Phase 6 — Merge Separate Admin Code

If admin was started in a separate project:

```
cd "BIHE Website"
git checkout -b merge-admin-panel

# Option A: Copy folders manually
cp -r "../bihe-admin/src/app/admin" "./src/app/"
cp -r "../bihe-admin/src/app/api" "./src/app/"
cp -r "../bihe-admin/src/components/admin" "./src/components/"

# Option B: Merge Git history
git remote add admin-repo <ADMIN_REPO_URL>
git fetch admin-repo
git merge admin-repo/main --allow-unrelated-histories
```

Then fix:

```
npm install
npm run build
```

Move duplicate types into `src/lib/types/content.ts`.

---

8. Phase 7 — Testing Checklist

#	Test	Expected Result
1	Open `/`	Public site loads
2	Open `/admin` without login	Redirects to login
3	Login as admin	Dashboard opens
4	Add faculty in admin	Saves to database
5	Open `/b-com`	New faculty appears
6	Edit announcement	Top bar updates
7	Logout admin	`/admin` blocked again
8	Run `npm run build`	No errors

---

9. Phase 8 — Deployment

Step 18: Deploy on Vercel (One Project)

1. Push `main` branch to GitHub
2. Import repository at vercel.com
3. Add environment variables:
   - `DATABASE_URL`
   - `NEXTAUTH_SECRET`
   - `NEXTAUTH_URL`
4. Deploy

Live URLs:

URL	Purpose
https://bihe.edu	Public website
https://bihe.edu/admin	Admin panel
https://bihe.edu/api/*	API endpoints

---

10. Phase 9 — Team Rules (Always)

Daily Rules for Both Developers

1. Pull `main` every morning before coding
2. Never edit the same file on the same day without coordinating
3. Shared types only in `src/lib/types/`
4. Admin team never edits `src/lib/*-content.ts` directly
5. Website team never edits `src/app/admin/` or write API logic
6. All merges via Pull Request — no direct push to `main`
7. Commit `.env.example` (not `.env.local`):

```
DATABASE_URL=
NEXTAUTH_SECRET=
NEXTAUTH_URL=http://127.0.0.1:3000
ADMIN_EMAIL=
ADMIN_PASSWORD=
```

Message to Share With Teammate

> 1. *Clone the same repo: [repo URL]*
> 2. *Open that folder in Cursor (same workspace root)*
> 3. *Work in `src/app/admin/` and `src/app/api/` only*
> 4. *Use shared types in `src/lib/types/`*
> 5. *Do not edit `src/lib/*-content.ts` — admin will replace that with API data*
> 6. *Pull before starting each day; push to your branch; merge via PR*

---

11. Week-by-Week Plan

Week 1

Who	Tasks
Both	Git repo + shared types + branch rules
Admin	Database + auth + `/admin` shell + announcements API
Website	Keep building pages + add fetch helpers

Week 2

Who	Tasks
Admin	Faculty CRUD + programs CRUD + image upload
Website	Wire `/b-com` + announcement bar to API

Week 3

Who	Tasks
Admin	News + PDF admin
Website	Wire news, courses, remove static fallbacks

Week 4

Who	Tasks
Both	Test + fix bugs + deploy to Vercel

---

12. Quick Reference

Who Owns What

Area	Owner
Public pages & UI	Website team
Admin UI	Admin team
API routes	Admin team
Database	Admin team
Shared types	Both (coordinate via PR)
Content fetch layer	Website team

Content to Make Dynamic First

Priority	Content	Current File
High	B.Com programme table	`b-com-admin-content.ts`
High	Faculty lists	`b-com-admin-content.ts`, ICC, etc.
High	Announcement bar	`AnnouncementBar.tsx`
High	News / events	`NewsEventsSection.tsx`
Medium	Course details	`courses-content.ts`
Medium	PDF documents	`public/documents/`
Lower	Full page paragraphs	many `*-content.ts`

Integration Options Compared

Approach	Best For
One repo + `/admin` routes	BIHE (recommended)
Monorepo (`apps/web` + `apps/admin`)	Larger teams
Headless CMS (Sanity, Payload)	Non-technical editors
Two separate repos	Temporary only — merge later

Document prepared for BIHE development team.

For questions, coordinate via Git Pull Requests and shared `src/lib/types/` contracts.